

Undercroft

A self-hosted LLM inference platform built to demonstrate production-grade deployment practices on constrained infrastructure. Ollama sits behind a rate-limited FastAPI gateway, deployed on k3s via GitOps (Argo CD), with automated vulnerability scanning, sealed secrets, network isolation, and Prometheus/Grafana observability — all running on a self-provisioned Oracle Cloud compute instance.

Why this exists

The goal wasn't "a model behind an API" — it was building the CI/CD, security, and operational scaffolding that makes a service safe to run and safe to change.

Architecture

```
git push (main)
-> GitHub Actions (lint, test, Trivy scan, build)
-> GHCR (container registry)
-> Argo CD detects the change, auto-syncs the cluster
-> k3s (undercroft namespace):
    gateway - FastAPI, API-key auth, concurrency limiting, /metrics
    ollama  - CPU inference, model persisted on a PersistentVolumeClaim
    NetworkPolicy restricts ollama ingress to the gateway only
    Prometheus scrapes gateway /metrics -> Grafana dashboard
```

Components

- [gateway/](#) — FastAPI service in front of Ollama: API key auth, request concurrency limiting (protects the CPU-only inference backend from overload), Prometheus metrics, health check used by k8s probes and deploy gating
- [k8s/base/](#) — Deployment/Service manifests for gateway + Ollama, a NetworkPolicy restricting Ollama ingress to the gateway only, and a ServiceMonitor so Prometheus scrapes the gateway automatically
- [k8s/argocd/](#) — Argo CD Application manifest — apply once, then [git push](#) is the deploy mechanism from then on
- [.github/workflows/ci.yml](#) — lint -> test -> build -> Trivy scan (fails on HIGH/CRITICAL vulns) -> push to GHCR -> bump manifest for GitOps sync

Security

- API key is stored as a Bitnami Sealed Secret — only ciphertext ever lives in git; it's decryptable only by this cluster's private key
- Every image is scanned with Trivy before it can reach GHCR
- A NetworkPolicy is the only thing allowed to reach the inference backend inside the cluster

Observability

Prometheus scrapes the gateway's `/metrics` endpoint every 15s; a Grafana dashboard ("Undercroft Gateway") tracks request rate and queue depth in real time.

Load test results

50 requests, 5 concurrent, gateway concurrency limit of 2, CPU-only inference:

- Median latency: ~14s, max: ~15-16s, zero dropped connections
- A controlled comparison (full monitoring stack vs. scaled to zero) showed no meaningful latency difference — the bottleneck is CPU-bound inference itself, not observability overhead. This is what the concurrency limiter exists for: queueing requests gracefully instead of letting unbounded concurrent inference overwhelm the CPU.

Build plan

See [docs/PLAN.md](#) for the original day-by-day build plan and [docs/LOG.md](#) for the full dev log, including real bugs hit and fixed along the way.

Local dev

```
cd gateway
pip install -r requirements.txt
export OLLAMA_URL=http://localhost:11434
export GATEWAY_API_KEY=dev-key
uvicorn main:app --reload
```

Scope notes

Progressive delivery (canary rollout via Argo Rollouts) was designed for but not implemented given the timeline — see [docs/PLAN.md](#) for the reasoning and what a v2 would add.